

R3.04 – Qualité de développement

3 – Classes et objets en Java

Sébastien Thon

BUT Informatique 2ème année
IUT d'Aix-Marseille Université, site d'Arles
Département Informatique

1. Déclarer une classe

Java est un langage très fortement orienté objet, basé sur des classes. Comme en C++, une classe en Java est composée de :

Données membres

- Type simple: **char**, **int**, **double**, ...
- Type composé: **String**, autres objets, tableaux, listes...

Méthodes

- Fonctions permettant de manipuler les données.

Exemple : une classe Cercle

- Données membres caractérisant un cercle :
 - rayon
 - cx, cy : position du centre
- Définir le comportement du cercle :
 - Cercle()
 - déplacer()
 - modifieRayon()
 - périmètre()

```
public class Cercle
{
    // DONNEES DE LA CLASSE Cercle :
    public int cx, cy;    // position du centre
    public int rayon;    // rayon

    // METHODES DE LA CLASSE Cercle :

    // Constructeur de la classe :
    // Comme en C++, cette méthode porte le même nom que la classe
    // ATTENTION : pas de type de retour, pas même 'void'
    public Cercle(int x, int y, int r)
    {
        cx = x;
        cy = y;
        rayon = r;
    }
}
```

Code à écrire dans un fichier
portant le même nom que la
classe, donc **Cercle.java**

```
// Affichage des données de la classe
public void afficheInfos()
{
    System.out.println("Centre en " + cx + "," + cy);
    System.out.println("Rayon : " + rayon);
}
```

```
//Calcul du périmètre d'un cercle
public double périmètre()
{
    return 2 * Math.PI * rayon;
}
```

```
// Déplace le centre du cercle en (x, y)
public void déplacer(int x, int y)
{
    cx = x;
    cy = y;
}
```

```
// Modifie la valeur courante du rayon
public void modifieRayon(int r)
{
    rayon = r;
}
```

```
// Échange la position d'un cercle avec celle
// du cercle passé en paramètre
public void échange(Cercle autre)
{
    int tmp;

    tmp = cx;           // échanger la position en cx
    cx = autre.cx;
    autre.cx = tmp;

    tmp = cy;           // échanger la position en cy
    cy = autre.cy;
    autre.cy = tmp;
}

} // Fin de la classe Cercle
```

2. Constructeur

Les constructeurs sont utilisés pour initialiser correctement les données d'un objet au moment de la création de l'objet en mémoire. Le constructeur est une méthode portant le **même nom** que la classe.

```
Cercle C = new Cercle() ;
```

Cercle() est le constructeur par défaut. Il initialise :

- les variables numériques à 0 ou 0.0
- Les caractères à '\0'
- Null pour les objets (chaînes de caractères, etc.)

Surcharge de méthode

Il est possible de surcharger n'importe quelle méthode d'une même classe, c'est à dire d'avoir plusieurs méthodes portant le même nom.

La méthode à utiliser est déterminée en regardant la liste des paramètres attendus par chaque méthode.

Exemple :

```
public class Cercle
{
    public void modifier (int nx)
    public void modifier (int nx, int ny)
    public void modifier (int nx, int ny, int nr)
```

Ainsi, il est possible de définir plusieurs constructeurs pour une même classe.

→ **Surcharge** du constructeur

```
public Cercle(int x, int y, int r)
{
    cx = x;
    cy = y;
    rayon = r;
}
```

```
public Cercle(int x, int y)
{
    cx = x;
    cy = y;
}
```

3. Définir un objet

1) Déclarer un objet

```
NomClasse  nomObjet;
```

Exemple:

```
Cercle  objet_cercle;
```

Cette déclaration crée une case mémoire nommée **objet_cercle**.

Cette case contiendra l'adresse de l'espace mémoire où seront stockées les informations concernant **objet_cercle**, mais pour le moment cet objet n'a pas encore été créé.

2) Réserver l'espace mémoire à l'aide de l'opérateur new

```
// réserver de l'espace mémoire pour l'objet  
nomObjet = new nomClasse(paramètres) ;
```

Exemple :

```
Cercle objet_cercle;  
objet_cercle = new Cercle(10, 20, 5) ; // cx, cy, rayon
```

Remarque : on peut aussi faire les deux opérations en une seule ligne :

```
Cercle objet_cercle = new Cercle(10, 20, 5) ;
```

Remarques :

new réserve suffisamment d'espace mémoire pour stocker les données de la classe, et détermine l'adresse où sera stocké l'ensemble de ces informations.

Lors de la réservation, les données de la classe sont initialisées.

L'opérateur new appelle le constructeur de la classe.

L'objet défini est un représentant de la classe :

objet_cercle est une **instance** de la classe **Cercle**.

Écrire la classe suivante dans un fichier `FaireDesCercles.java`, et le placer dans le même répertoire que `Cercle.java`.

```
public class FaireDesCercles
{
    public static void main(String [] arg)
    {
        Cercle A = new Cercle(10,10,2);
        A.afficheInfos();
    }
}
```

```
double p = A.périmètre() ;  
System.out.print("Périmètre du cercle:" + p) ;
```

```
A.déplacer(5, 2) ;  
System.out.println("Après déplacement:") ;  
A.afficheInfos() ;
```

```
A.modifieRayon(10) ;  
System.out.println("Après agrandissement:") ;  
A.afficheInfos() ;
```

```
Cercle B = new Cercle();  
B.cx = 50;  
B.cy = 50;  
B.rayon = 100;  
System.out.println("Le cercle B:");  
B.afficheInfos();
```

```
B.échange(A);
```

```
System.out.println("Après échange le cercle A:");  
A.afficheInfos();  
System.out.println("et le cercle B:");  
B.afficheInfos();
```

```
}
```

```
}
```


4. Passage de paramètres par référence

Les **types simples** passés en paramètre à une méthode sont passé par **valeur**

→ impossible de les modifier dans la méthode.

```
void methode1(int val)
{
    val = 2;
}
```

```
int variable = 1;
methode1(variable);
System.out.print(variable); // affiche : 1
```

Les **objets** passés en paramètre à une méthode sont forcément passés par **référence** → on peut les modifier.

```
void methode2(Cercle c)
{
    c.rayon = 3;
}
```

```
Cercle unCercle = new Cercle(10, 20, 1);    // xc, yc, rayon
methode2(unCercle);
System.out.println(unCercle.rayon);          // affiche : 3
```

5. Encapsulation des données d'une classe

Pour des questions de contrôle, de sécurité, on ne doit pas pouvoir modifier des objets sans utiliser les méthodes prévues à cet effet.

Exemple de la classe Cercle :

- La modification des coordonnées du centre doit se faire avec la méthode **deplacer()**:

```
unCercle.deplacer(2,4);
```

- Pour l'instant on peut aussi directement modifier les coordonnées du centre :

```
unCercle.cx=2; unCercle.cy=4;
```

Comme en C++, il y a en Java 3 niveaux de protection des données :

- ***public*** : accessible par tous les objets de l'application.
- ***private*** : accessible seulement par les membres de la classe ou par des objets de la même classe.
- ***protected*** : accessible seulement par les membres de la classe ou par les membres d'une sous-classe (voir la notion d'héritage).

Par défaut, c'est le type **public** qui est employé.

Exemple :

```
public class CerclePrivé
{
    private int xc, yc, rayon ;

    public void modifieRayon(int r)
    {
        rayon = r;
    }
}
```

// Code dans une autre classe :

```
CerclePrivé unCercle = new CerclePrivé(1,2,3) ;
unCercle.rayon = 10; // ERREUR, donnée privée
System.out.print( unCercle.rayon ); // ERREUR, donnée privée
unCercle.modifieRayon(10); // OK, méthode publique
```

6. Héritage

Concept permettant de créer de nouvelles classes en réutilisant les fonctionnalités d'une autre classe, en lui apportant des variations.

Syntaxe :

```
Class B extends A
{
    // données et méthodes de la classe B
}
```

B est une **sous-classe** de A (B = classe dérivée)

A est une **super-classe** (A = classe de base, classe mère)

Exemple :

Création d'une classe **Forme** contenant des propriété et traitements géométriques basiques.

→ Peut être réutilisée pour la classe **Cercle**, mais aussi **Rectangle**, **Trapèze**, etc.

→ Possibilité d'ajouter de nouveaux comportements supplémentaires à la classe **Forme**.

→ Pour déterminer si une classe B **Hérite** d'une classe A, il suffit de savoir s'il existe une relation « **est un** » entre B et A. Un cercle « est une » forme géométrique.

```

public class Forme
{
    protected int x, y, couleur;

    public Forme(int nx, int ny)
    {
        x = nx ;
        y = ny ;
        couleur = 0;
    }

    public void afficher()
    {
        System.out.println("Position en " + x + "," + y);
        System.out.println("Couleur : " + couleur);
    }

    public void déplacer(int nx, int ny)
    {
        x = nx;
        y = ny;
    }
} // Fin de la classe Forme

```


Utilisation :

```
public class Cercle extends Forme
{
    private int rayon;

    public Cercle(int xx, int yy)
    {
        super(xx, yy);
        couleur = 10;
        rayon = 0;
    }

    public void afficher()
    {
        super.afficher();
        System.out.println("Rayon : " + rayon);
    }
}
```

Le constructeur d'une classe héritée

Les classes dérivées possèdent leur propre constructeur.

Le constructeur de la classe **Cercle** appellera le constructeur de la classe **Forme**.

S'il n'y a pas de constructeur, le compilateur cherche le constructeur par défaut (sans paramètre) de la classe supérieure (**Forme**).

Attention : problème s'il n'y a pas de constructeur sans paramètre dans la classe supérieure.

On peut appeler directement un constructeur de la classe mère depuis le constructeur de la classe avec le mot clé **super** :

```
public Cercle(int xx, int yy)
{
    super(xx, yy);
    couleur = 10;
    rayon = 0;
}
```

@override

Si dans une classe dérivée on redéfinit une méthode d'une classe de base, on peut faire précéder cette méthode par le mot clef **@override** :

```
public class Cercle extends Forme
{
    ...
    // On suppose qu'on a une méthode tracer() dans Forme
    @override
    public void tracer(Graphics g)
    {
        g.drawOval(x-rayon/2, y-rayon/2, rayon*2, rayon*2);
    }
}
```

Cette annotation sert pour deux choses :

- pour la génération de documentation avec Javadoc.
- le compilateur vérifiera que dans la classe de base il y a bien une méthode qui a ce prototype.

@override permet au compilateur de vérifier l'authenticité de la surcharge, ce qui permet d'éviter les erreurs de frappe :

```
public class Parent {  
    public void doThat() {...}  
}
```

```
public class Child extends Parent {  
    @Override  
    public void doTaht() {...}  
}
```

→ provoquera une erreur de compilation car **doTaht** n'existe pas dans la classe **Parent**.

7. Le mot clé ***static***

- Si le mot- clé ***static*** est placé devant une variable ou une méthode :
 - ⇒ Réservation d'un seul et unique emplacement en mémoire.
 - ⇒ Espace accessible pour tous les objets du même type.
- Si le mot- clé ***static*** n'apparaît pas
 - ⇒ Réservation, à chaque appel de l'opérateur ***new***, d'un espace mémoire.

```
public class Cercle
{
    private int xc, yc, rayon;    // position du centre et rayon
    public static int nombre;    // nombre de cercles

    public Cercle(int x, int y, int r)
    {
        xc = x;
        yc = y;
        rayon = r;
        nombre = nombre + 1;
    }
}
```

xc, yc, rayon sont des **variables d'instance**.

nombre est une **variable de classe**. C'est un espace mémoire commun à tous les objets de la même classe.

→ il n'est donc pas nécessaire de créer d'instance d'une classe (d'objet) pour accéder à une variable ou une méthode déclarée **"static"**, on peut y accéder directement **via le nom de la classe**.

```
public class CompterDesCercles
{
    public static void main(String [] arg)
    {
        Cercle A = new Cercle(10,10,5) ;
        System.out.println("Nombre de cercles : " + Cercle.nombre) ;

        Cercle B = new Cercle(20,20,3) ;
        System.out.println("Nombre de cercles : " + Cercle.nombre) ;
    }
}
```